LAB 1:FOUNDATIONS OF MICROSERVICES ARCHITECTURES

Element : Microservice-oriented Architectures

Professor : Zouheir BANOU Group : Master Big Data & Data Science

Instructions: For each case, answer every question before selecting an architecture. You may defend a different choice if your reasoning addresses all stated constraints. Do not assume microservices are inherently superior.

Decision rule: Your final defense must include (a) the chosen architecture, (b) three case-specific supporting facts, (c) why the other two are less suitable, (d) one drawback of your choice, and (e) one trigger for revisiting it.

Case 1 — The Neighborhood Bookstore

A local bookstore launches a website for 200-400 customers. Three developers build and maintain Authentication, Catalog, Orders, Payments, and Inventory. The catalog changes weekly; purchases are infrequent. The business uses one application server and one relational database. Orders must update stock immediately and reliably. The team releases once every two weeks, has no dedicated operations staff, and must launch in six weeks.

1. Which two business requirements are most critical at launch? Cite the scenario.
2. Do different modules require independent scaling or independent deployment? What evidence supports your answer?
3. How tightly must Orders, Payments, and Inventory coordinate? What is gained by keeping operations within one application?
4. What additional costs or failure modes would microservices introduce for this team?
5. Would clearer internal code organization still be beneficial even if deployment remains unified? Explain.
6. Choose one architecture. Defend it against both alternatives.

Case 2 — The Growing Retailer

A retailer now serves 30,000 customers and has a team of 12 developers. Authentication, Catalog, Orders, Payments, and Inventory share one codebase. Frequent changes to Catalog have begun breaking Order logic because modules access each other's tables directly. Teams need clearer ownership and internal interfaces. Traffic is moderate and similar across modules. All components can still be deployed together during a weekly maintenance window. The company has limited operational expertise; Order placement and stock reservation need strong consistency.

1. Identify the main architectural problem: traffic, deployment, or coupling. Quote two symptoms.
2. Define five functional modules and assign one clear responsibility to each.
3. For each dependency (Orders→Inventory, Orders→Payments, Catalog→Inventory), specify a permitted internal interface rather than direct table access.
4. Does the case justify independent network deployments? Compare likely benefits with added complexity.
5. What boundaries and data-access rules would reduce accidental coupling while retaining one deployment unit?

6. Draw a proposed architecture showing modules, internal interfaces, and deployment boundary.
7. Choose an architecture. Defend it against both alternatives, identify a disadvantage, and name a measurable migration trigger.

Case 3 — The National Marketplace

A marketplace operates across several regions and has 150 engineers organized into independent Catalog, Orders, Payments, Identity, and Inventory teams. Catalog traffic increases ×20 during sales while Payments rises only ×2. Catalog updates occur daily, Payments changes monthly under strict verification, and Identity has its own security release schedule. A Catalog failure should not prevent existing orders from being processed where possible. The organization has mature monitoring, automated deployment, and engineers experienced with distributed systems. Teams need separate data ownership and deployment schedules.

1. Which constraints indicate a need for independently deployable business capabilities? Identify at least three.
2. Which components have materially different scaling needs? What would scaling the entire application waste?
3. Propose service boundaries for Catalog, Orders, Payments, Identity, and Inventory; explain one boundary that might reasonably remain combined.
4. Draw service-to-service API interactions for placing an order. Mark which service owns each relevant data set.
5. Identify two new distributed-system risks and one mitigation for each (e.g., timeouts, partial failure, cross-service consistency).
6. Why would a single deployable monolith or modular monolith be less suitable for the stated organization?
7. Choose an architecture, defend it with evidence, acknowledge a drawback, and specify a condition for consolidating services.

Case 4 — The Regional Company

A regional e-commerce company has been operating for five years and currently serves approximately 100,000 customers. Its application includes Authentication, Product Catalog, Orders, Payments, and Inventory, all developed in Java and deployed as a single application.

The company employs 25 developers divided into three teams. Although each team is responsible for specific functionalities, developers frequently modify code belonging to other teams. Changes to the Product Catalog have occasionally caused unexpected failures in Orders.

During promotional campaigns, Catalog traffic increases by up to 10 times, while Orders and Payments experience only moderate increases. However, these campaigns occur just four times per year, and the existing infrastructure can handle normal traffic without difficulty.

The company releases updates every two weeks. Management would like to deploy Catalog updates more frequently, but Payments and Inventory require careful coordination to maintain data consistency. The company is also planning to introduce a Python-based AI recommendation system next year.

The technical team has experience maintaining Java applications but limited experience managing distributed systems. Management has approved a moderate infrastructure budget and expects the architecture to support business growth over the next three years.

Question: Which architecture would you recommend for this company: Monolithic, Modular Monolithic, or *Microservices?*